

Quantum Network-Based Prediction of Cancer Driver Genes

Patrícia Marques,¹ Andreas Wichert,^{1,2} and Bruno Coutinho^{3,4}

¹*Instituto Superior Técnico, University of Lisbon, Lisboa, Portugal*

²*INESC-ID, GAIPS, Lisboa, Portugal*

³*Instituto de Telecomunicações, Lisboa, Portugal*

⁴*German Aerospace Center (DLR), Cologne, Germany*

(Dated: July 31, 2025)

Identification of cancer driver genes is fundamental for the development of targeted therapeutic interventions. The integration of mutational profiles with protein-protein interaction (PPI) networks offers a promising avenue for the detection of such driver genes [1, 2]. However, the application of quantum computing methodologies in this domain remains largely unexplored. In this study, we introduce a supervised quantum framework for cancer driver gene prediction that combines mutation scores with network topology via a novel state preparation scheme, *Quantum Multi-order Moment Embeddings* (QMME), motivated by the classical method of Gumpinger et al [3]. QMME encodes low-order statistical moments over the mutation scores of a node’s immediate and second-order neighbors, and encodes this information into quantum states. These are used as inputs to a kernel-based quantum binary classifier that discriminates known driver genes from others. Through a simulation on a real PPI network, we confirm that this approach achieves competitive predictive performance relative to classical algorithms baselines. In a brief complexity analysis, we identify the potential of our approach in uncovering a quantum speedup for quantum network-based prediction of cancer driver genes. This work underscores the potential of supervised quantum graph learning frameworks to advance biological discovery.

I. INTRODUCTION

A central objective in oncology is to identify and catalogue genes underlying human cancer [4–6]. While genes with high mutation rates are likely drivers, most *cancer driver genes* have few mutations. In these cases, frequency-based methods alone cannot reliably distinguish rare cancer driver genes from irrelevant ones [5].

One explanation lies in the networked nature of cellular processes: genes operate within pathways and complexes, so cancer may result from disruption at the pathway level rather than mutations in specific individual genes. Recent research has embraced this interaction-based perspective of cancer, integrating biological networks with statistical measures of gene–cancer associations to identify novel candidate cancer driver genes [1, 7]. In these networks, nodes correspond to genes and edges represent relationships between them. Protein–protein interaction (PPI) networks [8–10] are a widely used representation of gene interactions, as they integrate information from diverse data sources, tissues, and molecular processes.

Previous studies have combined protein–protein interaction networks with gene-level cancer association scores, like the MutSig value [11], using clustering or network propagation methods. NetSig, for instance, focuses on the local neighborhood of each gene to identify cancer driver genes while correcting for node degree bias [1].

A few studies have explored supervised approaches, incorporating existing knowledge of known driver genes during prediction. Gumpinger et al. used annotations from the Cancer Gene Census (CGC) to train classifiers for discovering new cancer driver genes [3]. This method models the task as a node (supervised) classification problem in a

molecular interaction network, combining network topology with mutation-score features. It achieved more than a 10% improvement in area under the precision–recall curve (AUPRC) over baseline methods.

Network-based approaches derive their strength from integrating weak mutation signals across connected genes within molecular interaction networks, rather than relying on strong statistical evidence from any individual gene. Their success underscores the value of graph-structured representations in cancer driver gene discovery. However, as biological networks grow in size and complexity, traditional methods face challenges in scalability and representational efficiency. This motivates the exploration of quantum algorithms as a promising alternative for encoding and processing such structured data, including protein–protein interaction networks. Quantum computing, and in particular quantum machine learning (QML), has been proposed as a tool with potential for inference tasks [12–15].

Classical methods often rely on local statistical descriptors, such as neighborhood means or low-order moments, to capture structural information in graphs [3]. However, quantum embedding strategies that incorporate such statistics remain unexplored. Moreover, many existing QML approaches neglect the gate complexity of state preparation, limiting their practicality as end-to-end quantum solutions. Motivated by these gaps, we introduce a quantum protocol that embeds low-order statistical moments of features in the one-hop and two-hop neighborhoods into amplitude-encoded quantum states.

In this work, we introduce the *Quantum Multi-order Moment Embedding* (QMME) protocol, which encodes these local graph statistics into quantum states suitable

for inference. Using this embedding, we design an almost-fully quantum pipeline for node-level binary classification on graph-structured data with feature-labeled nodes. Classification is performed using the Swap Test classifier [16], a non-parametric, distance-based quantum kernel method that distinguishes embeddings via quantum interference.

The entire process, from embedding to inference, is executed within the quantum domain, requiring no classical optimization or training. It relies only on minimal data access assumptions and limited classical post-processing. The pipeline explicitly addresses state preparation (practical circuit construction), ensuring that embeddings can be constructed with logarithmic qubit resources and polynomial-depth circuits.

We validate our method through numerical simulations on real/synthetic graphs, analyzing classification performance alongside quantum resource metrics such as circuit depth and query complexity. This work establishes a proof-of-principle for end-to-end quantum learning on graph-structured data and lays the groundwork for quantum models on complex network inference tasks.

II. PROBLEM SETUP

A. Notation

Let $\mathcal{G} = (V, E, \mathbf{x})$ be an unweighted, undirected graph with $|V| = N$ nodes, where $V = [N]$ denotes the set of node indices and $E \subseteq V \times V$ is the edge set. The connectivity of $\mathcal{G}$ is described by its adjacency matrix $A \in \{0, 1\}^{N \times N}$, where $A_{vu} = 1$ indicates the presence of an edge between nodes v and u , and $A_{vu} = 0$ its absence.

$$\mathcal{N}_v^1 = \{u \in V \mid A_{vu} = 1\}. \quad (1)$$

We assume $\mathcal{N}_v^1$ is ordered by increasing node index. This allows us to define the function $r(v, \ell)$ as

$$r(v, \ell) := \mathcal{N}_v^1(\ell), \quad \ell \in \{0, \dots, |\mathcal{N}_v^1| - 1\}. \quad (2)$$

That is, it returns the index of the ℓ -th neighbor of v in $\mathcal{N}_v^1$. Equivalently, $r(v, \ell)$ is the column index of the ℓ -th nonzero entry in row v of A . If $\ell \geq |\mathcal{N}_v^1|$, the function returns a flag indicating so.

The two-hop neighborhood $\mathcal{N}_v^2 \subseteq V$ is defined as

$$\mathcal{N}_v^2 = \{w \in V \mid \exists u \in V, A_{vu} = 1 \wedge A_{uw} = 1\}. \quad (3)$$

Each node $v \in V$ has a scalar feature value $x_v \in \mathbb{R}$. The collection of all node features is denoted by the vector

$$\mathbf{x} = (x_1, x_2, \dots, x_N) \in \mathbb{R}^N. \quad (4)$$

A protein-protein interaction (PPI) network can be represented as a graph, where nodes are genes and edges indicate interactions. Each gene's association with cancer

can be quantified using the MutSig P-value [11], which reflects statistical significance based on mutational burden, mutations clustering, and functional impact [3].

B. Node Embeddings

Define the function $\varphi(X)$ that maps a scalar random variable $X \sim \mathcal{P}$ to its first four moments.

$$\varphi(X) = [\mathbb{E}_X[X], \mathbb{E}_X[X^2], \mathbb{E}_X[X^3], \mathbb{E}_X[X^4]]. \quad (5)$$

In practice, these expectations are estimated using empirical moments computed from a finite sample. Let the sample $\mathbf{s} = [s_1, \dots, s_k]$ of X denote a realization of the random variable X , where each $s_j \in \mathcal{R}$. We define the *origin moment embedding* of $\mathbf{s}$ as

$$\varphi(\mathbf{s}) = \left[\sum_{j=1}^k \frac{s_j}{k}, \sum_{j=1}^k \frac{s_j^2}{k}, \sum_{j=1}^k \frac{s_j^3}{k}, \sum_{j=1}^k \frac{s_j^4}{k} \right]. \quad (6)$$

This maps a finite sample $\mathbf{s}$ of X to a vector capturing its first four sample origin moments.

The hop order $c \in \{1, 2\}$ specifies the neighborhood distance from node v . Given $\mathbf{x}$ defined in eq. (4),

$$\mathcal{X}_v^c = \{x_u \in \mathbf{x} \mid u \in \mathcal{N}_v^c\}, \quad (7)$$

notes the set of feature values of nodes at distance c from v , which are samples drawn from distribution $\mathcal{P}_v^c$. The origin moment embedding function $\varphi(\cdot)$ is applied to each $\mathcal{X}_v^c$, such that

$$\mathbf{m}_v^c := \varphi(\mathcal{X}_v^c) \in \mathbb{R}^4. \quad (8)$$

For vertex $v \in V$, we define its embedding as the concatenation of $\mathbf{m}_v^c$ from the one-hop and two-hop neighborhoods:

$$\mathbf{m}_v = [\varphi(\mathcal{X}_v^1), \varphi(\mathcal{X}_v^2)] \in \mathbb{R}^8. \quad (9)$$

This defines the *node embedding function*

$$\mathbf{m}_v : V \rightarrow \mathbb{R}^8. \quad (10)$$

C. Problem Statement

We assume a supervised binary classification setting on a graph $\mathcal{G} = (V, E, \mathbf{x})$, where a subset of nodes $V_l \in V$ is labeled with $y_v = 1$ for $v \in V_l$ and $y_v = 0$ for $v \in V_0 \subset V_l$. Our goal is to predict the label y_v for any node $v \in V$, using a method that combines a quantum *node embedding* $U_{\mathcal{G}}$, based on the node features $\mathbf{x}$ and the structure of $\mathcal{G}$, with a quantum binary classifier $\mathcal{C}$. This

combination should enable the prediction of whether any node v belongs to class 1 or class 0. That is,

$$\mathcal{C} \left(U_{\mathcal{G}} |v\rangle |0\rangle^{\otimes m} \right) = \hat{y}_v, \quad (11)$$

where $\hat{y}_v$ denotes the predicted label of node v .

More specifically, our goal is to develop a quantum graph-based node embedding model for the supervised task of identifying cancer driver genes. We consider a protein-protein interaction (PPI) network with per-gene mutation scores, used as node features in the graph.

III. QUANTUM MULTI-ORDER MOMENT EMBEDDING (QMME)

The adjacency matrix A of a graph is generally non-unitary and thus cannot be directly implemented as a quantum operation. A common approach to encode such non-unitary matrices into quantum circuits is through *block-encoding*. Under this model, we assume access to a unitary operator $U_{\mathcal{G}}$ that acts as follows:

$$U_{\mathcal{G}} |v\rangle |0\rangle^{\otimes m} = |v\rangle |\psi_v\rangle, \quad (12)$$

where $|\psi_v\rangle$ contains a *neighborhood-moment* component, $|\psi_m\rangle$, and an *orthogonal* component, $|\psi_{\perp}\rangle$. Specifically, $|\psi_m\rangle$, in our case, will be given by

$$|\psi_m\rangle = |\mathbf{c} \odot \mathbf{m}_v\rangle \otimes |0\rangle^{\otimes (m-3)}, \quad (13)$$

where $\mathbf{c} \odot \mathbf{m}_v$ denotes the element-wise (Hadamard) product between the constant vector $\mathbf{c} \in \mathbb{R}^8$ and the mutation feature vector $\mathbf{m}_v \in \mathbb{R}^8$.

A. Quantum Input Model

A commonly input model is s -sparse, assuming there are at most $s = 2^s$ nonzero entries in each row of the matrix [17]. We assume access to the following quantum (unitary) oracles acting on appropriate Hilbert spaces:

$$O_L : |v\rangle |\ell\rangle \mapsto |v\rangle |r(v, \ell)\rangle, \quad (14)$$

$$\tilde{O}_X : |v\rangle |0^p\rangle \mapsto |v\rangle |\tilde{x}_v\rangle, \quad (15)$$

$$\tilde{O}_K : |v, c\rangle |0^p\rangle \mapsto |v, c\rangle |\tilde{k}_v^c\rangle, \quad (16)$$

where v is the node index, ℓ the neighbor index, $\tilde{x}_v$ the d' -bit fixed point representation of the node feature value x_v , and $\tilde{k}_v^c$ the binary representation of the node degree k_v^c . The operator $\oplus$ denotes the bitwise XOR operation.

We also assume access to the controlled versions of oracles O_L , $\tilde{O}_X$, and $\tilde{O}_K$ and their inverses (which act on them).

The implementation of the oracles may be challenging, and we will only consider the query complexity with respect to them.

Note that the oracle O_K can be deduced from O_L by querying the number of neighbors (degree) of node

B. Oracles O_X and O_{K-1}

We define the feature rotation operator as

$$F_X : |0\rangle |\tilde{x}_v\rangle \mapsto \left(x_v |0\rangle + \sqrt{1 - |x_v|^2} |1\rangle \right) |\tilde{x}_v\rangle, \quad (17)$$

and the inverse-degree rotation operator as

$$F_{K-1} : |0\rangle |\tilde{k}_v\rangle \mapsto \left(\frac{1}{k_v^c} |0\rangle + \sqrt{1 - \left| \frac{1}{k_v^c} \right|^2} |1\rangle \right) |\tilde{k}_v\rangle. \quad (18)$$

These operators are implemented using controlled rotation gates, specifically multi-controlled R_y gates. For F_X , the ancilla rotation angle is $\theta(\tilde{x}_v) = 2 \arcsin(x_v)$, and for F_{K-1} , it is $\theta(\tilde{k}_v) = 2 \arcsin(1/k_v^c)$.

$$a : |i\rangle^{\otimes 5} \begin{array}{c} |i\rangle \\ |0\rangle^{\otimes 5} \\ |v\rangle \end{array} \xrightarrow{O_X} \begin{array}{c} |i\rangle \\ (x_v |0\rangle + \sqrt{1 - |x_v|^2} |1\rangle)^{\otimes i} |0\rangle^{\otimes 4-i} \\ |v\rangle \end{array}$$

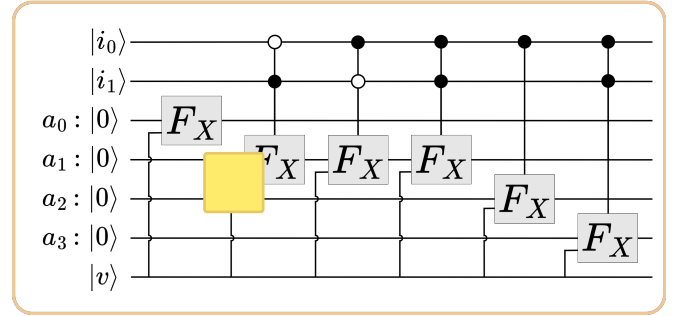


FIG. 2. Oracle O_X action in detail. The implementation of O_X requires the usage of a work register with d' -qubits, that has been omitted here, but explained below.

The feature oracle O_X action depends on the control register $|i\rangle$, where i determines how many times F_X is applied:

$$|v, i\rangle |0\rangle^{\otimes 5} \xrightarrow{\tilde{O}_X} |v, i\rangle |0\rangle |\tilde{x}_v\rangle \quad (19)$$

$$\xrightarrow{C-F_X} |v, i\rangle \left(x_v |0\rangle + \sqrt{1 - |x_v|^2} |1\rangle \right)^{\otimes i} |0\rangle^{\otimes 4-i} |\tilde{x}_v\rangle \quad (20)$$

$$\xrightarrow{\tilde{O}_X^{-1}} |v, i\rangle \left(x_v |0\rangle + \sqrt{1 - |x_v|^2} |1\rangle \right)^{\otimes i} |0\rangle^{\otimes 5-i}. \quad (21)$$

The 2-bit control register $|i\rangle$ determines the number of ancilla qubits to which the feature rotation operation F_X is applied. There are four copies of F_X , applied

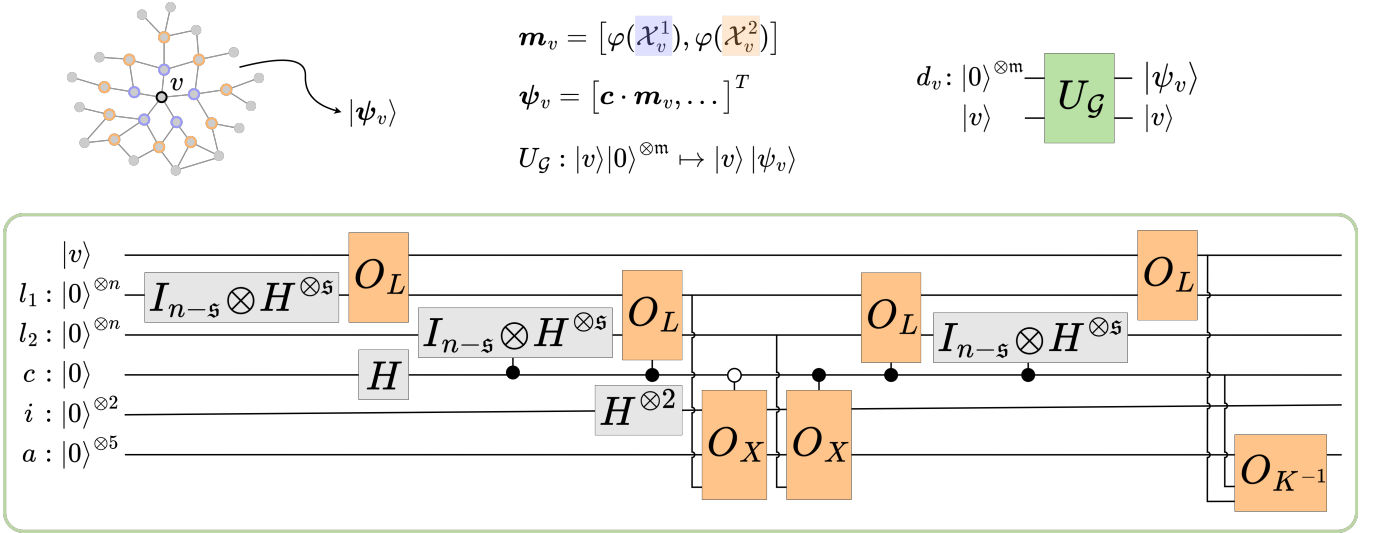


FIG. 1. Quantum Multi-order Moment Embedding (QMME) circuit scheme. The unitary operator U_G performs amplitude encoding by mapping $|v\rangle \otimes |0\rangle^{\otimes m} \mapsto |v\rangle \otimes |\psi_v\rangle$, where $|v\rangle$ encodes the node index $v \in V$ and $|0\rangle^{\otimes m}$ is an initialized zero register with $m = O(n)$. The output state $|\psi_v\rangle \in \mathbb{R}^{2^m}$ is a quantum embedding capturing the graph $\mathcal{G} = (V, E, \mathbf{x})$ neighborhood statistics of v , based on the origin moment embedding function from eq. (10). The input data include synthetic graphs or real-world PPI networks, with scalar node features, i.e., mutation scores.

conditionally based on $|i\rangle$: for $i = 0$, only the first rotation is applied; for $i = 1$, the first two; for $i = 2$, the first three; and for $i = 3$, all four. Each gate is triggered by a specific pattern in $|i\rangle$, illustrated in fig. 2.

In fig. 1, we omit the intermediate register $|\tilde{x}_v\rangle$, and represent the oracle simply as:

$$O_X : |0^{\otimes 4}\rangle |i\rangle |v\rangle \mapsto \left(x_v |0\rangle + \sqrt{1 - |x_v|^2} |1\rangle \right)^{\otimes i} |0\rangle^{\otimes 4-i} |v\rangle \quad (22)$$

Specifically,

$$\langle 0^{\otimes 4} | O_X | 0^{\otimes 4} \rangle |i\rangle |v\rangle = x_v^i |0\rangle |i\rangle |v\rangle. \quad (23)$$

The inverse-degree oracle O_{K-1} is implemented as follows:

$$|v, c\rangle |0\rangle |0\rangle \xrightarrow{\bar{O}_K} |v, c\rangle |0\rangle |\tilde{k}_v\rangle \quad (24)$$

$$\xrightarrow{F_{K-1}} |v, c\rangle \left(\frac{1}{k_v^c} |0\rangle + \sqrt{1 - \left| \frac{1}{k_v^c} \right|^2} |1\rangle \right) |\tilde{k}_v\rangle \quad (25)$$

$$\xrightarrow{\bar{O}_K^{-1}} |v, c\rangle \left(\frac{1}{k_v^c} |0\rangle + \sqrt{1 - \left| \frac{1}{k_v^c} \right|^2} |1\rangle \right) |0\rangle \quad (26)$$

In fig. 1, we omit the intermediate register $|\tilde{k}_v\rangle$, and represent the oracle simply as:

$$O_{K-1} : |0\rangle |c\rangle |v\rangle \mapsto \left(\frac{1}{k_v^c} |0\rangle + \sqrt{1 - \left| \frac{1}{k_v^c} \right|^2} |1\rangle \right) |c\rangle |v\rangle. \quad (27)$$

C. QMME Feature Map

Let the QMME unitary operator (or feature map) be

$$U_G : |v\rangle |0\rangle^{\otimes m} \mapsto |v\rangle |\psi_v\rangle, \quad (28)$$

where m is the number of qubits in the data register, fig. 3. In particular, this includes the c qubit to encode the one or two-hop register, the i two-qubit register corresponding to the index of the i -th moment.

The state vector $|\psi_v\rangle$ decomposes into a *neighborhood-moment* component, $|\psi_m\rangle$, and an *orthogonal* component, $|\psi_\perp\rangle$:

$$|\psi_v\rangle = |\psi_m\rangle + |\psi_\perp\rangle \quad (29)$$

$$|\psi_m\rangle = |c \odot \mathbf{m}_v\rangle |0\rangle^{\otimes (m-3)} \quad (30)$$

$$= \sum_{c \in [2]} \frac{1}{2\sqrt{2} s^c} \sum_{i \in [4]} \varphi_i(\mathcal{X}_v^c) |c, i\rangle \otimes |0\rangle^{\otimes (m-3)}, \quad (31)$$

$$|\psi_\perp\rangle = \sum_{a \neq 0} |\psi_{\perp, a}\rangle |a\rangle. \quad (32)$$

$$|\psi_v\rangle = \sum_{j=0}^7 \left(c_j m_{v,j} |0\rangle^{\otimes (m-3)} + |\perp\rangle \right) |j\rangle \quad (33)$$

$$= |c \odot \mathbf{m}_v\rangle |0\rangle^{\otimes (m-3)} + |\perp\rangle, \quad (34)$$

where, omitting the work registers, we get $m = 2n + 8$, but in general $m = O(n)$, as in eq. (28).

Let us call the base case where $i = 0$ and $c = 0$, and we aim to retrieve $m_{v,1} = \varphi_1(\mathcal{X}_v^1)$. For this, the inner product structure is given by

$$\langle c = 0, i = 0, 0^{\otimes(m-3)} | U_G | v \rangle | 0 \rangle^{\otimes m} \quad (35)$$

$$\begin{aligned} &= \frac{1}{2\sqrt{2}s k_v^0} \sum_{l,l'} x_{r(v,l)} \delta_{r(v,l),r(v,l')} \\ &= \frac{1}{2\sqrt{2}s k_v^c} \sum_l x_{r(v,l)} \\ &= \frac{1}{2\sqrt{2}s} \sum_{u \in \mathcal{N}_v^1} \frac{x_u}{|\mathcal{N}_v^1|} \\ &= \frac{1}{2\sqrt{2}s} \varphi(\mathcal{X}_v^1), \end{aligned} \quad (36)$$

with U_G the QMME operator, k_v^c the the number of c -hop neighbors of v , and s the adjacency matrix sparsity. The scalar x_j denotes the feature value indexed by j , while the function $r(v,l)$ returns the l -th neighbor of node v . The Kronecker delta $\delta_{r(v,l),r(v,l')}$ equals 1 if the two neighbors coincide and 0 otherwise.

This expression naturally generalizes to arbitrary indices v, c, i , yielding

$$\langle c, i | \langle 0 |^{\otimes 5+2s} U_G | v \rangle | 0 \rangle^{\otimes m} = \frac{\varphi_i(\mathcal{X}_v^i)}{2\sqrt{2}s^c}. \quad (37)$$

To characterize the state structure more explicitly, consider the computational basis $\{|i\rangle\}_{i=0}^7$ of the main register. Satisfies

$$\langle 0^{(5+2m)} | \perp \rangle = 0, \quad (38)$$

which guarantees a clean separation between the *moment* component (associated with the ancilla zero state) and the *orthogonal* component (supported on the orthogonal subspace).

Step-by-step derivation of the application of U_G , the QMME unitary, on an input state is described in Figure 4.

IV. QUANTUM CLASSIFICATION FRAMEWORK

The QMME protocol will be applied to both the register encoding the superposition of the labeled nodes indeces, and to the register encoding the test node index.

The Swap Test computes the overlap between two quantum states. This computation yields $|\langle \psi | \phi \rangle|^2$ for two pure states $|\psi\rangle$ and $|\phi\rangle$, and more generally gives $\text{Tr}(\rho\sigma)$ for two (possibly mixed) states ρ and σ . In quantum optics, the Swap Test has a simple physical implementation, not so obvious for gate-based quantum computers, such as superconducting ones from IBM and Google, and the IonQ's trapped-ion quantum computer [18].

Our quantum classification is illustrated in fig. 3. The first step, state preparation, includes applications of the

U_G embedding unitary for both test and labeled data. The final state is an application of the Swap Test classifier [16].

We adapt the framework via balancing. To address imbalances in the training labels, instead of the Hadamard gate, one can initialize the index qubits of the labeled nodes (v_l) with an equal number of positive and negative nodes. This balancing acts on the sum of inner products as expressed in Equation (9) of the original paper, ensuring that label prevalence does not disproportionately influence classification outcomes.

Additionally, we incorporated label-dependent weighting factors w_m designed to correct for label parity discrepancies. These weights satisfy the normalization condition

$$\sum_{l_v=0} w_{v|l_v=0} = \sum_{l_v=1} w_{v|l_v=1}, \quad (39)$$

and explicitly account for the number of training nodes N_0, N_1 in each class. This weighting strategy, inspired by the theoretical formulation but rarely emphasized in prior work, substantially improved classification performance in our Quantum Metric Mean Embedding plus Quantum Classifier (QMME+QC) setting.

The full pipeline of fig. 3 shows the quantum circuit which evolves the state as

$$|0\rangle |0\rangle^{\otimes m} |v_t\rangle |0\rangle |0\rangle^{\otimes m} |0\rangle^{\otimes n} \quad (40)$$

$$\mapsto \sum_{v_l \in V_l} |0\rangle |0\rangle^{\otimes m} |v_t\rangle |0\rangle |0\rangle^{\otimes m} |v_l\rangle \quad (41)$$

$$\mapsto |0\rangle |\psi_t\rangle |v_t\rangle |0\rangle |\psi_v\rangle |0\rangle^{\otimes n} \quad (42)$$

$$\mapsto |0\rangle |\psi_t\rangle |v_t\rangle |y_v\rangle |\psi_v\rangle |0\rangle^{\otimes n} \quad (43)$$

$$\text{Swap Test} \quad (44)$$

V. NUMERICAL SIMULATIONS

[ongoing]

Classical simulations of quantum circuits. performance against state-of-the-art classical baselines.

A. Dataset Description

B. Results

Finally, to validate our code, we replicated the simplest Swap Test example from the reference paper, involving two nodes each with two features. This test confirmed the correctness of our Swap Test implementation and helped isolate errors unrelated to core classification logic.

Experiments with label imbalance showed that equal weighting of nodes across classes generally yields the anticipated classification results. However, the introduction of differential weights w_m to compensate for label parity

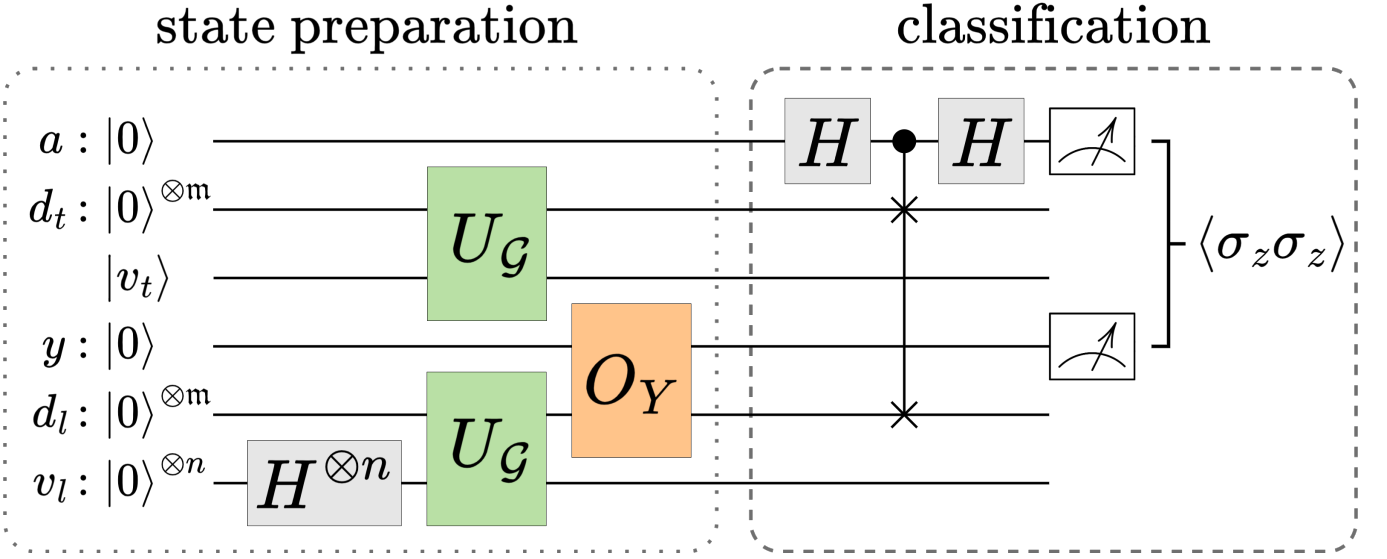


FIG. 3. Overview of the full pipeline with the QMME protocol for node classification. The Swap Test classifier on top of QMME state preparation. The first register corresponds to the ancilla qubit (a), the second the test data qubits (d_t), the third the index qubits of the test node (v_t), the fourth stores the label qubit (y), the fifth the labeled data qubits (d_l), and the sixth the index qubits of the labeled nodes (v_l). The operator U_G , together with the Hadamard gate and the O_Y oracle, are applied to prepare the input quantum state necessary for the classification procedure. The state is then processed by a quantum classifier $\mathcal{C}$ to predict node labels $\hat{y}_v \in \{0, 1\}$. The classification outcome is extracted via a Swap Test and two-qubit measurement statistics on the ancilla and label registers. Measurements are repeated for each unlabeled node, either sequentially or in parallel.

markedly improved classification confidence and accuracy. Some unexpected deviations appeared in more intricate examples, likely due to the nonlinear influence of polynomial kernels embedded in the Swap Test procedure.

Having a balanced-label dataset was critical to balancing contributions from positive and negative classes, effectively mitigating biases arising from dominant label frequencies.

VI. COMPLEXITY ANALYSIS

We analyze the complexity of the QMME circuit in terms of gate count, circuit depth, and required qubits. Let $N = 2$ be the number of labeled nodes and d' the precision (in bits) of the encoded features and degrees.

Loading a feature vector via $\tilde{O}_X$ maps $|v\rangle |0\rangle \mapsto |v\rangle |\tilde{x}_v\rangle$ and can be implemented with $O(d')$ gates using structured memory access or qRAM. The rotation F_X applies a controlled $R_y(2 \arccos x_v)$ gate, with θ computed on-the-fly via quantum arithmetic, contributing $O(\text{poly } n + d')$ gates. The oracle $\tilde{O}_{K-1}$ and F_{K-1} follow the same complexity.

The total number of oracle calls per QMME state preparation is constant, with two calls to each oracle (and its inverse), plus up to five amplitude-encoding operations. The number of Hadamard gates scales as $O(n + \mathfrak{s})$, and the total gate count is dominated by arithmetic and oracle logic.

Work qubits include l for input registers, $\mathfrak{s}$ for neighbors

basis-encoding, and $O(1)$ for control. The total number of qubits is thus $O(n + l + \mathfrak{s})$, where d' depends on precision.

Metric	Complexity Estimate
Gate count	$O(\text{poly}(n) + l)$
Circuit depth	$O(\text{poly}(n) + l)$
Work qubits	$O(d' + \mathfrak{s})$
Total qubits	$O(n + d' + \mathfrak{s})$

[be very clear on constraints of current quantum hardware]

VII. DISCUSSION AND CONCLUSIONS

QMMEP realizes a fully quantum node-classification pipeline - no classical optimization loops; all encoding, embedding, and inference occur quantumly. We demonstrate polynomial resource scaling and solid performance on synthetic data. Future work includes extensions to deeper moments, weighted graphs, and implementation on near-term hardware.

An architecture that others can build on (generalizable moment-encoding circuit).

Potential improvements or extensions include multi-class classification (either gene categories or even outside the scope of biological networks), and mention applications to other types of nets.

[mention limitations] [adrese o porqu  de escolher momentos at  ordem 4. e de como cada parte do circuito

(1-hop, 2-hop) contribui para a performance]

-
- [1] H. Horn, M. S. Lawrence, C. R. Chouinard, Y. Shrestha, J. X. Hu, E. Worstell, E. Shea, N. Ilic, E. Kim, A. Kamburov, *et al.*, *Nature methods* **15**, 61 (2018).
 - [2] M. Nourbakhsh, K. Degn, A. Saksager, M. Tiberti, and E. Papaleo, *Briefings in Bioinformatics* **25**, bbad519 (2024).
 - [3] A. C. Gumpinger, K. Lage, H. Horn, and K. Borgwardt, *Bioinformatics* **36**, i508 (2020).
 - [4] L. A. Garraway and E. S. Lander, *Cell* **153**, 17 (2013).
 - [5] B. Vogelstein, N. Papadopoulos, V. E. Velculescu, S. Zhou, L. A. Diaz Jr, and K. W. Kinzler, *science* **339**, 1546 (2013).
 - [6] G. M. Frampton, A. Fichtenholtz, G. A. Otto, K. Wang, S. R. Downing, J. He, M. Schnall-Levin, J. White, E. M. Sanford, P. An, *et al.*, *Nature biotechnology* **31**, 1023 (2013).
 - [7] M. A. Reyna, M. D. Leiserson, and B. J. Raphael, *Bioinformatics* **34**, i972 (2018).
 - [8] K. Lage, E. O. Karlberg, Z. M. Størling, P. I. Olason, A. G. Pedersen, O. Rigina, A. M. Hinsby, Z. Tümer, F. Pociot, N. Tommerup, *et al.*, *Nature biotechnology* **25**, 309 (2007).
 - [9] T. Li, R. Wernersson, R. B. Hansen, H. Horn, J. Mercer, G. Slodkiewicz, C. T. Workman, O. Rigina, K. Rapacki, H. H. Stærfeldt, *et al.*, *Nature methods* **14**, 61 (2017).
 - [10] D. Szklarczyk, A. L. Gable, D. Lyon, A. Junge, S. Wyder, J. Huerta-Cepas, M. Simonovic, N. T. Doncheva, J. H. Morris, P. Bork, *et al.*, *Nucleic acids research* **47**, D607 (2019).
 - [11] M. S. Lawrence, P. Stojanov, C. H. Mermel, J. T. Robinson, L. A. Garraway, T. R. Golub, M. Meyerson, S. B. Gabriel, E. S. Lander, and G. Getz, *Nature* **505**, 495 (2014).
 - [12] P. Wittek, *Quantum machine learning: what quantum computing means to data mining* (Academic Press, 2014).
 - [13] M. Schuld, I. Sinayskiy, and F. Petruccione, *Contemporary Physics* **56**, 172 (2015).
 - [14] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, *Nature* **549**, 195 (2017).
 - [15] D. K. Park, C. Blank, and F. Petruccione, *Physics Letters A* **384**, 126422 (2020).
 - [16] C. Blank, D. K. Park, J.-K. K. Rhee, and F. Petruccione, *npj Quantum Information* **6**, 41 (2020).
 - [17] L. Lin, arXiv preprint arXiv:2201.08309 (2022).
 - [18] L. Cincio, Y. Subaşı, A. T. Sornborger, and P. J. Coles, *New Journal of Physics* **20**, 113022 (2018).

$$\begin{aligned}
|v, 0^{\otimes}\rangle &\xrightarrow{I_{n+3} \otimes H^{\otimes s} \otimes I_{s+5}} \frac{1}{\sqrt{s}} |v, 0^{\otimes 3}\rangle \sum_{\ell_1 \in [s]} |\ell_1, |0^{\otimes s+5}\rangle\rangle \\
&\xrightarrow{O_L} \frac{1}{\sqrt{s}} |v, 0^{\otimes 3}\rangle \sum_{\ell_1 \in [s]} |r(v, \ell_1), |0^{\otimes s+5}\rangle\rangle \\
&\xrightarrow{I_n \otimes H \otimes I_{2+s+5}} \frac{1}{\sqrt{2s}} |v\rangle \sum_{c \in [2]} \sum_{\ell_1 \in [s]} |c, 0^{\otimes 2}, r(v, \ell_1), |0^{\otimes s+5}\rangle\rangle \\
&\xrightarrow{I_{n+3+s} C-H^{\otimes s} \otimes I_5} \frac{1}{\sqrt{2s}} |v\rangle \sum_{\ell_1 \in [s]} \left(|0_c, 0^{\otimes 2}, r(v, \ell_1), |0^{\otimes s}\rangle\rangle + \frac{1}{\sqrt{s}} \sum_{\ell_2 \in [s]} |1_c, 0^{\otimes 2}, r(v, \ell_1), \ell_2\rangle \right) |0^{\otimes 5}\rangle \\
&\xrightarrow{C-O_L} \frac{1}{\sqrt{2s}} |v\rangle \sum_{\ell_1 \in [s]} \left(|0_c, 0^{\otimes 2}, r(v, \ell_1), |0^{\otimes s}\rangle\rangle + \frac{1}{\sqrt{s}} \sum_{\ell_2 \in [s]} |1_c, 0^{\otimes 2}, r(v, \ell_1), r(r(v, \ell_1), \ell_2)\rangle \right) |0^{\otimes 5}\rangle \\
&\xrightarrow{I_{n+1} \otimes H^{\otimes 2} \otimes I_{2s+5}} \frac{1}{2\sqrt{2s}} |v\rangle \sum_{\ell_1 \in [s]} \sum_{i \in [4]} \left(|0_c, i, r(v, \ell_1), |0^{\otimes s}\rangle\rangle + \frac{1}{\sqrt{s}} \sum_{\ell_2 \in [s]} |1_c, i, r(v, \ell_1), r(r(v, \ell_1), \ell_2)\rangle \right) |0^{\otimes 5}\rangle \\
&\xrightarrow{C-O_X} \frac{1}{2\sqrt{2s}} |v\rangle \sum_{\ell_1 \in [s]} \sum_{i \in [4]} \left(x_{r(v, \ell_1)}^i |0_c, i, r(v, \ell_1), |0^{\otimes s}\rangle\rangle + \frac{1}{\sqrt{s}} \sum_{\ell_2 \in [s]} x_{r(r(v, \ell_1), \ell_2)}^i |1_c, i, r(v, \ell_1), r(r(v, \ell_1), \ell_2)\rangle \right) |0^{\otimes 5}\rangle + |v\rangle |\perp\rangle |0\rangle \\
&\xrightarrow{C-O_L} \frac{1}{2\sqrt{2s}} |v\rangle \sum_{\ell_1 \in [s]} \sum_{i \in [4]} \left(x_{r(v, \ell_1)}^i |0_c, i, r(v, \ell_1), |0^{\otimes s}\rangle\rangle + \frac{1}{\sqrt{s}} \sum_{\ell_2 \in [s]} x_{r(r(v, \ell_1), \ell_2)}^i |1_c, i, r(v, \ell_1), \ell_2\rangle \right) |0^{\otimes 5}\rangle + |v\rangle |\perp\rangle |0\rangle \\
&\xrightarrow{I_{n+3+s} C-H^{\otimes s} \otimes I_5} \frac{1}{2\sqrt{2s}} |v\rangle \sum_{\ell_1 \in [s]} \sum_{i \in [4]} \left(x_{r(v, \ell_1)}^i |0_c, i, r(v, \ell_1)\rangle + \frac{1}{\sqrt{s}} \sum_{x' \in \mathcal{X}_{r(v, \ell_1)}^1} x'^i |1_c, i, r(v, \ell_1)\rangle \right) |0^{\otimes s+5}\rangle + |v\rangle |\perp\rangle |0\rangle \\
&\xrightarrow{O_L} \frac{1}{2\sqrt{2s}} |v\rangle \sum_{\ell_1 \in [s]} \sum_{i \in [4]} \left(x_{r(v, \ell_1)}^i |0_c, i, \ell_1\rangle + \frac{1}{s} \sum_{x' \in \mathcal{X}_{r(v, \ell_1)}^1} x'^i |1_c, i, \ell_1\rangle \right) |0^{\otimes s+5}\rangle + |v\rangle |\perp\rangle |0\rangle \\
&\xrightarrow{I_{n+3} H^{\otimes s} \otimes I_{s+5}} \frac{1}{2\sqrt{2s}} |v\rangle \sum_{i \in [4]} \left(\sum_{x' \in \mathcal{X}_{r(v, \ell_1)}^1} x'^i |0_c\rangle + \frac{1}{s} \sum_{x'' \in \mathcal{X}_v^2} x''^i |1_c\rangle \right) |i, |0^{\otimes 2s+5}\rangle\rangle + |v\rangle |\perp\rangle |0\rangle \\
&\xrightarrow{O_{K-1}} \frac{1}{2\sqrt{2s}} |v\rangle \sum_{i \in [4]} \left(\sum_{x' \in \mathcal{X}_{r(v, \ell_1)}^1} x'^i |0_c\rangle + \frac{1}{s} \sum_{x'' \in \mathcal{X}_v^2} x''^i |1_c\rangle \right) \frac{1}{k_v^c} |i, |0^{\otimes 2s+5}\rangle\rangle + |v\rangle |\perp\rangle \\
&= |v\rangle \left(\sum_{i,c} (2\sqrt{2s}^{c+1})^{-1} \sum_{x''' \in \mathcal{X}_v^c} \frac{x'''^i}{k_v^c} |c, i, 0^{\otimes 2s+5}\rangle + |\perp\rangle \right) \\
&= |v\rangle \left(\sum_{i,c} (2\sqrt{2s}^{c+1})^{-1} \varphi(\mathcal{X}_v^c) |c, i, 0^{\otimes 2s+5}\rangle + |\perp\rangle \right) \\
&= |v\rangle \left(\sum_{j=0}^{j=7} c_j m_{v,j} |j, 0^{\otimes 2s+5}\rangle + |\perp\rangle \right)
\end{aligned}$$

FIG. 4. Step-by-step derivation of the application of U_G , the QMME unitary, on an input state. Quantum circuit transformation steps for computing node-neighborhood moment embeddings.